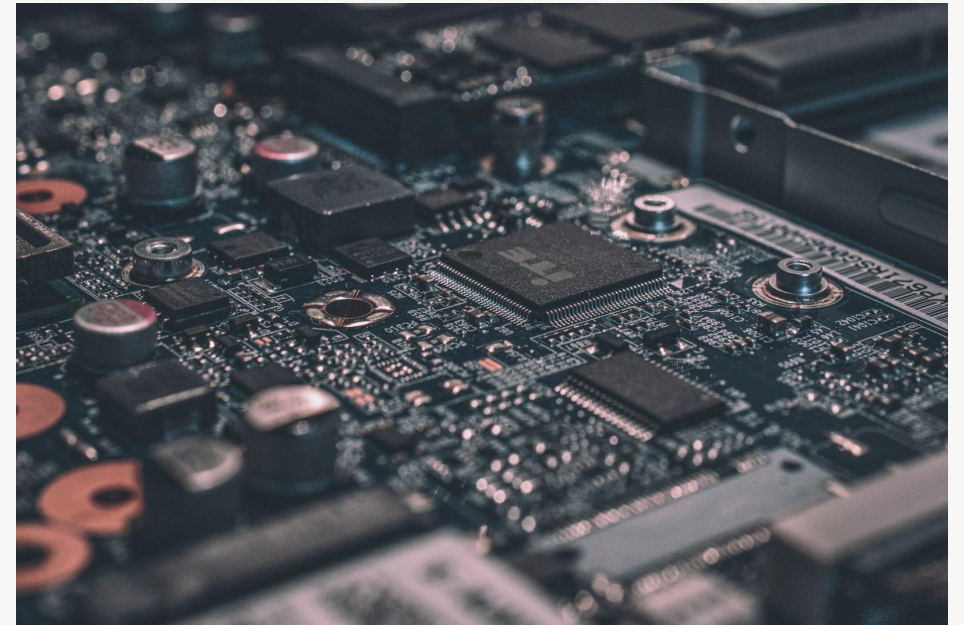


Mennyi fér el, és hogyan lesz gyorsabb?

Számábrázolás és párhuzamosság

Egész számok, adاتمéreték
és a számítás gyorsításának határai



A bitektől a több feldolgozóegységgel végzett munkáig.

Három kérdés vezet végig az órát

Mennyi fér el?

Véges számú bit.
Véges értéktartomány.
A túlcsordulás oka.

Mit jelentenek a bitek?

Előjel nélkül vagy előjellel.
Ugyanaz a bitsorozat más értéket jelenthet.

Mitől lesz gyorsabb?

Több tranzisztor.
Több feldolgozóegység.
A feladatot is fel kell osztani.

A számítógép képességeit a reprezentáció és a munka szervezése együtt határozza meg.

Egy bit: két lehetséges állapot

0 vagy 1

A bit egy bináris számjegy. Önmagában nem mondja meg, milyen adatot ábrázolunk.

Egy bájt: nyolc bit egy csoportban



1 bájt = 8 bit

8 biten 256 különböző bitsorozat létezik. A bájt nem feltétlenül egy karakter.

Kétféle többszörös: tízes és kettes alapú

Decimális jelölés	Érték bájtban	Bináris jelölés	Érték bájtban
1 kB	1 000	1 KiB	1 024
1 MB	1 000 000	1 MiB	1 048 576
1 GB	1 000 000 000	1 GiB	1 073 741 824
1 TB	10^{12}	1 TiB	2^{40}

A kB és a KiB különböző egység. A pontos jelölés tisztázza az értéket.

Miért látszik más méretűnek ugyanaz a meghajtó?

$$1 \text{ TB} \approx 931,3 \text{ GiB}$$

Ugyanazt a bájtmennyiséget két különböző egységben fejezzük ki.

A bit és a bájt felcserélése nyolcszoros különbség

Hálózati sebesség

100 Mbit/s

Másodpercenként
100 millió bit.

Adatátvitel bájtban

Elméletileg 12,5 MB/s.

$$100 \div 8 = 12,5$$

A tényleges átvitel kisebb is lehet.

Mbit/s és MB/s különböző jelölés: a nagy B bájtot jelent.

Gyors egységellenőrzés

80 Mbit/s elméletileg hány MB/s?

A: 10 MB/s

B: 80 MB/s

C: 640 MB/s

Előbb gondolkodjunk, utána ellenőrizzük közösen!

Előjel nélkül minden bitnek pozitív súlya van

1	0	1	0	0	1	0	1
128	64	32	16	8	4	2	1

$$128 + 32 + 4 + 1 = 165$$

Csak az 1-es bitekhez tartozó helyiértékeket adjuk össze.

n biten mennyi különböző érték fér el?

2^n különböző érték

Előjel nélkül a tartomány: $0 \dots 2^n - 1$.

Néhány fontos előjel nélküli tartomány

Bitszám	Értékek száma	Legkisebb ... legnagyobb
4 bit	16	0 ... 15
8 bit	256	0 ... 255
16 bit	65 536	0 ... 65 535
32 bit	2^{32}	0 ... 4 294 967 295
64 bit	2^{64}	0 ... $2^{64} - 1$

Több bit nagyobb tartományt ad, de minden rögzített méretnek van felső határa.

Mikor hasznos az előjel nélküli ábrázolás?

Nem negatív értékek

Bájtmenység.
Nyers adatbájt.
Bitmező és állapotjelzők.

A típusválasztás része

A tartomány illeszkedjen
a feladathoz.

A művelet közben is keletkezhet túl nagy
eredmény.

A nem negatív bemenet nem garantálja, hogy minden köztes eredmény is elfér.

A maximum vagy a darabszám?

Mi a legnagyobb előjel nélküli érték 8 biten?

A: 128

B: 255

C: 256

Előbb gondolkodjunk, utána ellenőrizzük közösen!

A negatív számhoz nincs külön mínuszjel a memóriában

A jelet is bitekkel ábrázoljuk.

Meg kell állapodnunk abban, hogyan értelmezzük a teljes bitsorozatot.

Előjel–abszolútérték: külön bit az előjelnek



1: negatív

0000101: az abszolútérték 5

10000101 → -5. Két nulla létezik: 00000000 és 10000000.

Egyes komplement: a pozitív minta bitfordítása

+5

0

0

0

0

0

1

0

1

-5

1

1

1

1

1

0

1

0

Ebben a módban is két nulla van: 00000000 és 11111111.

Kettes komplementens: bitfordítás és még egy lépés

+5

0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

Megfordítva

1	1	1	1	1	0	1	0
---	---	---	---	---	---	---	---

+1 → -5

1	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

Egyetlen nulla, és az összeadó áramkör az előjeles műveletekhez is használható.

Ugyanaz a -5 három különböző leírásban

Ábrázolás	-5 mintája 8 biten	Nullák száma
Előjel–abszolútérték	10000101	2
Egyes komplement	11111010	2
Kettes komplement	11111011	1

A bitsorozat és az ábrázolási mód együtt határozza meg az értéket.

A kettes komplementű tartomány nem szimmetrikus

$$-2^{n-1} \dots 2^{n-1} - 1$$

8 biten: $-128 \dots +127$. Az értékek száma itt is 256.

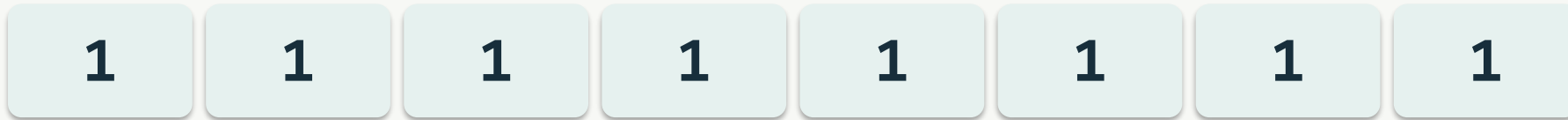
Miért jelent -5 -öt a 11111011?

1	1	1	1	1	0	1	1
-128	64	32	16	8	4	2	1

$$-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$$

A legfelső bit súlya kettes komplementben negatív.

Egy bájt kétféle értelmezése



Előjel nélkül

255

Kettes komplementben

-1

A típus és a feldolgozás szabályai adják meg, melyik jelentést használjuk.

Miért nincs +128 a nyolcbites előjeles tartományban?

Legkisebb érték

10000000 $\rightarrow$ -128

Ez önálló, érvényes kód.

Legnagyobb érték

01111111 $\rightarrow$ +127

A +128 már nem fér el
a nyolcbites előjeles tartományban.

A legkisebb negatív érték előjelváltása ugyanennyi biten túlcsoordulást okozhat.

Több bitre bővítéskor az előjelet is megőrizzük

-5, 8 biten **11111011**

-5, 16 biten **11111111 11111011**

Előjelkiterjesztés: a vezető előjelbitet ismételjük az új helyeken.

C# egész típusok: a névhez tartomány tartozik

Bitszám	Előjel nélküli típus	Előjeles típus
8	byte: 0 ... 255	sbyte: -128 ... 127
16	ushort: 0 ... 65 535	short: -32 768 ... 32 767
32	uint: 0 ... $2^{32} - 1$	int: $-2^{31} ... 2^{31} - 1$
64	ulong: 0 ... $2^{64} - 1$	long: $-2^{63} ... 2^{63} - 1$

A C# int 32 bites; a 64 bites operációs rendszer nem változtatja 64 bitessé.

A művelet nem korlátlan matematikai számokon fut

A gép oldalán

Regiszterek és aritmetikai egységek.

Véges szélességű operandusok és eredmények.

A program oldalán

A típus és a nyelv szabályai.

Mit teszünk, ha az eredmény kilép a tartományból?

A túlcsordulás kezelését a programozási nyelv és a végrehajtás is befolyásolja.

Előjel nélküli túlcsordulás: $15 + 1$ négy biten

15

1

1

1

1

+1

0

0

0

1

4 bites eredmény

0

0

0

0

A teljes összeg 10000_2 . Négy biten csak az alsó négy bit marad: 0.

Előjeles túlcsordulás: két pozitívból negatív?

+127

0

1

1

1

1

1

1

1

+1

0

0

0

0

0

0

0

1

8 bites minta

1

0

0

0

0

0

0

0

A matematikai +128 nem fér el. A megmaradt minta kettes komplementumban -128.

A kilépő átvitel és az előjeles túlcsordulás eltér

8 bites művelet	Kilépő átvitel	Előjelesen túlcsordul?
$11111111 + 00000001$	Igen	Nem: $-1 + 1 = 0$
$01111111 + 00000001$	Nem	Igen: $127 + 1$ nem fér el

Ugyanazt az összeadást előjel nélkül és előjellel eltérően értékelhetjük.

Kivonás: adjuk hozzá a negatív értéket!

+7

0

0

0

0

0

1

1

1

-5

1

1

1

1

1

0

1

1

Eredmény

0

0

0

0

0

0

1

0

$7 - 5 = 7 + (-5) = 2$. Ebben a példában az előjeles eredmény elér.

A szorzás is gyorsan kinőheti a tárolót

Matematikai eredmény

$$7 \times 3 = 21$$

$$21_{10} = 10101_2$$

Csak 4 bit marad

$$0101_2 = 5$$

Egy helyes műveleti algoritmus után is elveszhet az eredmény felső bitje.

Szorzás előtt a köztes eredmény szükséges tartományát is gondoljuk végig.

Egész osztás: az eredmény nem tört

Pozitív példa

$$7 / 2 = 3$$

$$7 \% 2 = 1$$

Hányados és maradék.

Negatív példa C#-ban

$$-7 / 2 = -3$$

$$-7 \% 2 = -1$$

Az egész osztás nulla felé csonkol.

$7 = 3 \times 2 + 1$. A nullával osztás hibát okoz.

Egy fontos C# részlet: byte + byte eredménye int

```
byte a = 250;  
byte b = 10;  
int sum = a + b;  
Console.WriteLine(sum);
```

Eredmény: 260

Az összeadás nagyobb típusban történik.

A byte tartományába visszaalakítás külön lépés.

Ugyanaz a konverzió: körbefordulás vagy kivétel?

```
int sum = 260;  
byte x = unchecked((byte)sum);  
// x értéke: 4  
  
byte y = checked((byte)sum);  
// OverflowException
```

A kontextus számít

unchecked: az alsó bitek maradnak.

checked: túlcsordulásakor hibajelzés.

A szélesebb tároló sem jelent korlátlan tartományt

```
int x = int.MaxValue;  
int y = unchecked(x + 1);  
  
Console.WriteLine(y);  
// -2147483648
```

A határ az int-nél is létezik

A matematikai eredmény
2 147 483 648 lenne.

Ez már kívül van az int
tartományán.

Melyik állítás helyes?

A számítás eredménye túl nagy a kiválasztott egész típushoz. Mi történhet?

**A: mindig helyes
marad**

**B: körbefordulhat
vagy hiba keletkezhet**

**C: automatikusan
long lesz**

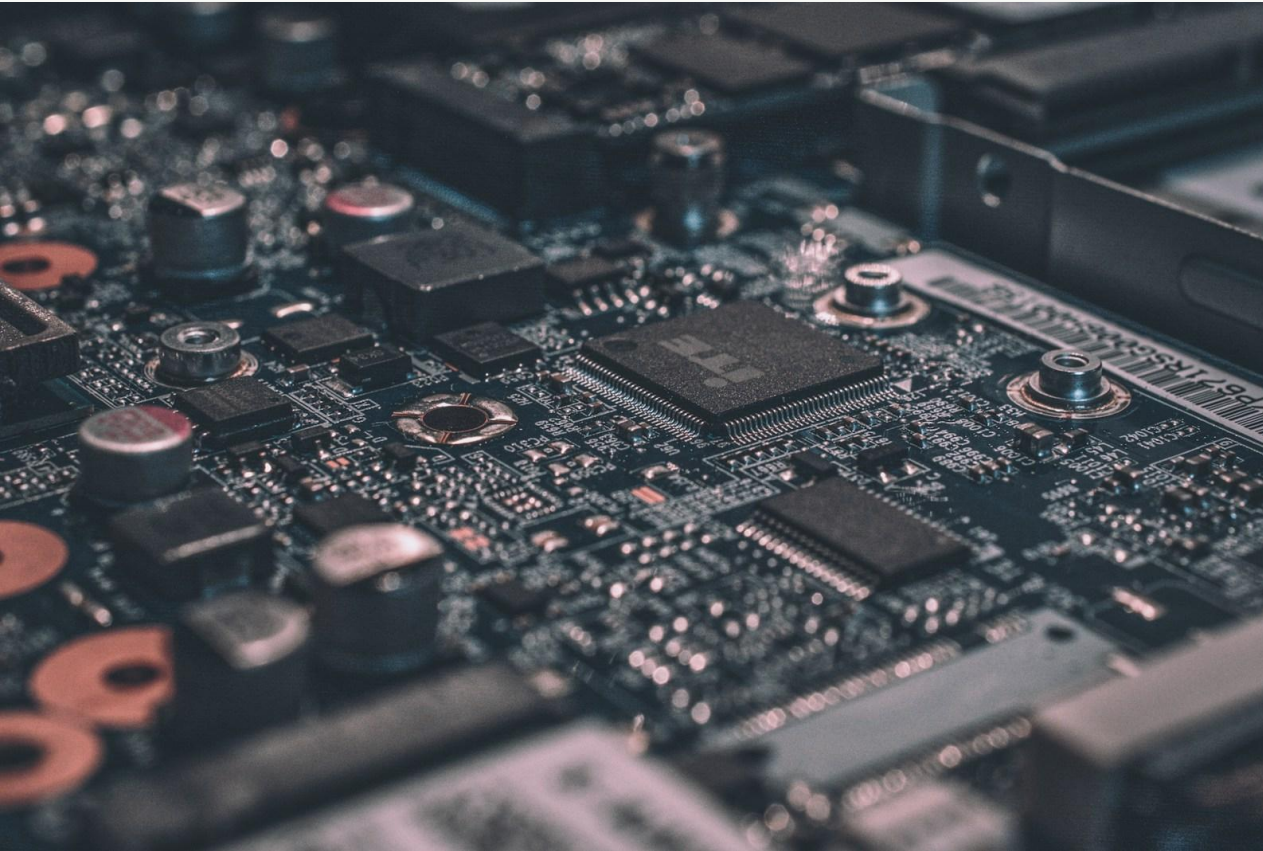
Előbb gondolkodjunk, utána ellenőrizzük közösen!

Rövid szünet, majd a gyorsítás kérdése

**Több bit: nagyobb tartomány.
Több mag: nagyobb sebesség?**

A második állítás csak megfelelő feltételek mellett igaz.

Moore megfigyelése: egyre több elem a chipen



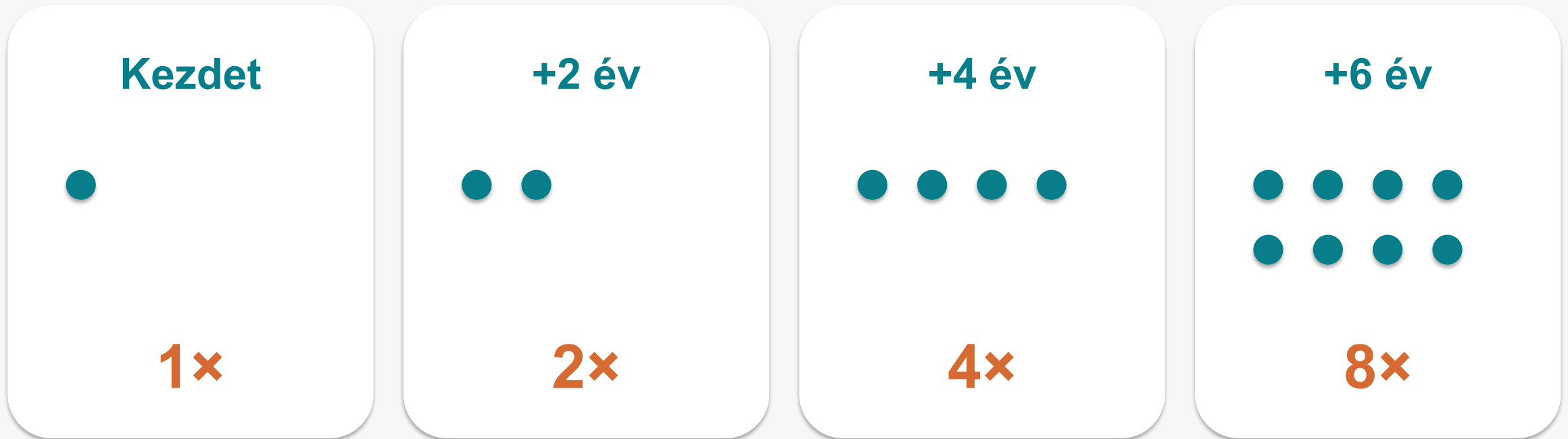
1965: egy iparági előrejelzés

Az integrált áramkörök egyre összetettebbek.

1975-től a közismert ütem: körülbelül kétfévenkénti duplázódás.

Illusztráció: Alexandre Debiève / Unsplash

A duplázódás sokszor ismétlődik



Szemléltető modell: relatív darabszám, nem konkrét termékadat vagy jövőbeli ígéret.

Több tranzisztor nem automatikusan kétszeres sebesség

Mire használhatók?

Több mag.
Nagyobb gyorsítótár.
Specializált egységek.

Mi korlátozható?

Hőtermelés és fogyasztás.
Adatmozgatás.
A program felépítése.

A tranzisztorszám, az órajel és egy program futási ideje három különböző mennyiség.

Technológiai szingularitás: egy vitatott jövőkép

Mi történik, ha a fejlődés önmagát gyorsítja?

Egyes elképzelések szerint a gyorsuló fejlődés után nehezebb lenne előre jelezni a társadalom változásait.

Háromféle állítás: érdemes megkülönböztetni

Megfigyelés

Moore:
a chip összetettségének
történeti növekedése.

Feltételes modell

Amdahl és Gustafson–
Barsis:
mit várhatunk adott
feltételekkel
a párhuzamosságtól?

Hipotézis

Szingularitás:
egy lehetséges jövőkép,
amely körül vita van.

A „törvény” szó nem minden témában ugyanazt a bizonyosságot jelenti.

Párhuzamosság: több egység dolgozik együtt



A feladatot részekre bontjuk

Több mag egy gépben.
Több gép egy rendszerben.

A munkát és az adatokat össze kell hangolni.

Fotó: Texas Advanced Computing Center, Frontera

Hétköznapi példa: közösen készítünk egy jelentést

Párhuzamosítható

Négy független fejezetet
négy ember dolgoz ki.

Mindenki saját részen dolgozik.

Soros lépés

A végső ellenőrzést
egy ember végzi.

Ezt a lépést a több szerző
önmagában nem rövidíti le.

A teljes időt a párhuzamos rész mellett a megmaradó soros rész is alakítja.

Amdahl: ugyanazt a munkát akarjuk hamarabb befejezni

1 feldolgozó



4 feldolgozó



Narancs: soros rész • Türkiz: párhuzamosítható rész

100 perc → 40 perc. A gyorsulás: $100 / 40 = 2,5\times$.

Amdahl tanulsága: a soros rész megmarad

Több feldolgozó

A 80 perces rész
egyre rövidebb lehet.

A 20 perces soros rész
nem tűnik el.

A modell határa

A teljes idő nem kerülhet
20 perc alá.

A gyorsulás legfeljebb
 $100 / 20 = 5\times$.

20% soros rész mellett az ideális gyorsulás felső határa $5\times$.

Amdahl képlete: a történet rövid leírása

$$S(N) = 1 / [s + (1 - s) / N]$$

s: a soros rész aránya az egyfeldolgozós futásban

N: a feldolgozók száma

S(N): az ideális gyorsulás

$$s = 0,2 \text{ és } N = 4 \rightarrow S = 1 / (0,2 + 0,8 / 4) = 2,5.$$

Mennyit hoz a következő néhány feldolgozó?

Feldolgozók száma	Ideális futási idő	Gyorsulás
1	100 perc	1×
2	60 perc	1,67×
4	40 perc	2,5×
8	30 perc	3,33×
16	25 perc	4×

A példa végig ugyanaz a 100 perces munka, benne 20 perc soros részzel.

Hány mag oldja meg a soros részt?

Ha a munka fele soros, mi az ideális gyorsulás felső határa?

A: 2×

B: 8×

C: nincs felső határ

Előbb gondolkodjunk, utána ellenőrizzük közösen!

Gustafson–Barsis: tegyünk fel másik kérdést!

**Ugyanannyi idő alatt
mekkora munkát végezhetünk?**

A több feldolgozót nagyobb feladat megoldására használjuk.

Négy feldolgozóval nagyobb feladat fér 100 percbe

4 feldolgozón

20 perc soros munka
+ 80 perc párhuzamos futás
= 100 perc

Ugyanez egy feldolgozón

20 perc soros munka
+ 4×80 perc munka
= 340 perc

A skálázott gyorsulás: $340 / 100 = 3,4\times$. Most a feladat mérete is nőtt.

Gustafson–Barsis képlete

$$S^G(N) = \alpha + N \cdot (1 - \alpha)$$

α : a soros idő aránya a párhuzamos futásban

N : a feldolgozók száma

$S^G(N)$: a skálázott feladat ideális gyorsulása

$$\alpha = 0,2 \text{ és } N = 4 \rightarrow S^G = 0,2 + 4 \cdot 0,8 = 3,4.$$

Két modell, két különböző kérdés

Összehasonlítás	Amdahl	Gustafson–Barsis
Mi marad rögzítve?	A feladat mérete	A párhuzamos futási idő
Mit növelünk?	A feldolgozók számát	A számukat és a feladatot
Mit kérdezzünk?	Mennyivel hamarabb kész?	Mennyivel több munka fér bele?
Soros idő aránya	Az egyfeldolgozós futásban	A párhuzamos futásban

A képletek értelmezéséhez először az összehasonlítás feltételeit tisztázzuk.

Miért lehet kisebb a valós gyorsulás?

Kommunikáció

Az egységek
adatot cserélnek.

Ez is időbe kerül.

Várakozás

Az egyik rész hamarabb
kész,
de várnia kell a másokra.

A terhelés nem egyenletes.

Közös erőforrás

A feldolgozók ugyanazt
a memóriát használják.

Az adatellátás szűk lehet.

Több feldolgozó önmagában nem garantál arányos gyorsulást.

Flynn: utasításfolyam és adatfolyam

Két kérdés: hány utasításfolyam, és hány adatfolyam?

	Egy adatfolyam	Több adatfolyam
Egy utasításfolyam	SISD	SIMD
Több utasításfolyam	MISD	MIMD

S = Single (egy) • M = Multiple (több) • I = Instruction • D = Data

SISD: egy utasításfolyam, egy adatfolyam



Szemléltető példa: egy soros program hagyományos végrehajtása.

SIMD: ugyanaz a művelet több adaton

Közös utasítás: adj hozzá 10-et!

1
↓

11

2
↓

12

3
↓

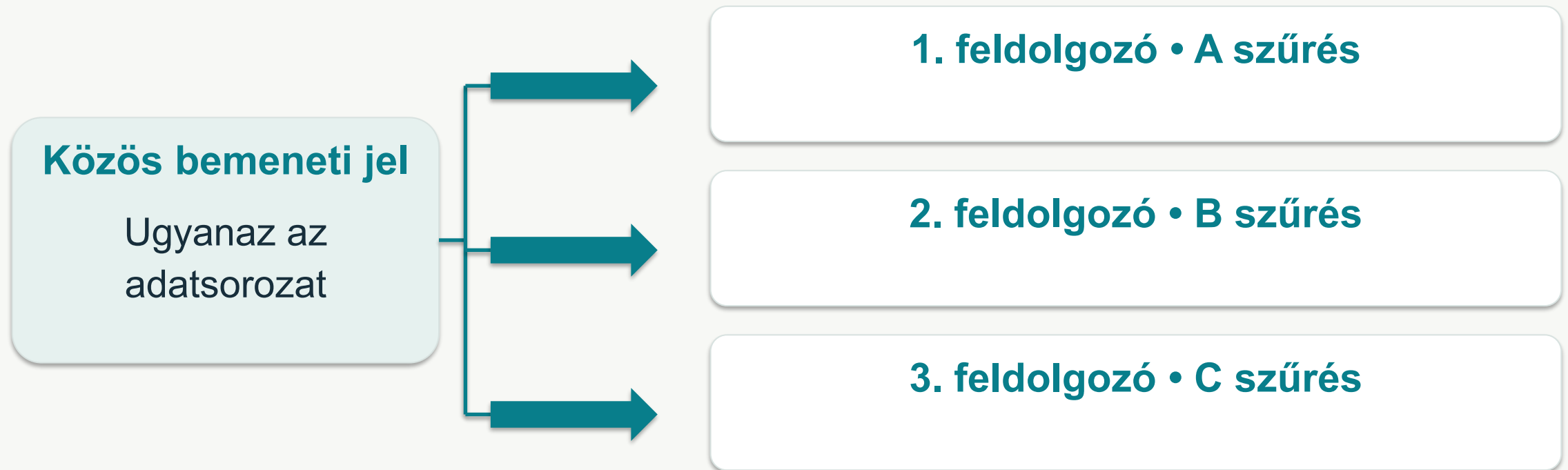
13

4
↓

14

Egy utasításfolyam irányítja a több adatot feldolgozó sávokat.

MISD: többféle művelet ugyanazon adatfolyamon



Ritka kategória; a különböző jelfeldolgozó szűrések elvi példaként szolgálnak.

MIMD: külön utasítások, külön adatok

1. feldolgozó

A feladat
A adatai

2. feldolgozó

B feladat
B adatai

3. feldolgozó

C feladat
C adatai

Példák: többmagos gépek és számítógép-klaszterek párhuzamos feladatai.

Egy mai gép többféle párhuzamosságot kombinálhat

1. processzormag

Saját utasításfolyam

SIMD

SIMD

SIMD

SIMD

2. processzormag

Saját utasításfolyam

SIMD

SIMD

SIMD

SIMD

A magok között MIMD, egy mag vektorutasításán belül SIMD is megjelenhet.

Melyik Flynn-kategóriára gondolunk?

Egy közös utasítás sok számtömb-elemen egyszerre ugyanazt az összeadást végzi.

A: SISD

B: SIMD

C: MIMD

Előbb gondolkodjunk, utána ellenőrizzük közösen!

Flynn négy kategóriája egy pillantásra

Típus	Utasításfolyam	Adatfolyam	Gondolati példa
SISD	Egy	Egy	Soros végrehajtás
SIMD	Egy	Több	Vektoros összeadás
MISD	Több	Egy	Közös jel eltérő szűrései
MIMD	Több	Több	Független párhuzamos feladatok

A besorolás alapja a folyamatok száma, nem pusztán a processzorok darabszáma.

Öt gondolat, amit érdemes hazavinni

Téma	Lényeg
Ábrázolás	Véges bitszámhoz véges értéktartomány tartozik.
Előjel	Ugyanaz a bitminta eltérő értéket jelenthet.
Műveletek	A típus és a túlcsordulás szabálya számít.
Gyorsulás	Először tisztázzuk: ugyanaz vagy nagyobb feladat?
Architektúra	Az utasítás- és adatfolyamok alapján osztályozunk.

Kilépőkártya: három rövid ellenőrzés

Bitsorozat

Mit jelent a 11111111
8 biten előjel nélkül,
és kettes
komplementsben?

Gyorsulás

Miért nem jelent
a kétszer annyi mag
mindig kétszeres
gyorsulást?

Architektúra

Mi a különbség
a SIMD és a MIMD
utasításfolyamai között?

A jó válasz rövid, de megnevezi a feltételeket is.

Források: mértékegységek és egész aritmetika

NIST: bináris előtagok és SI-mértékegységek

<https://physics.nist.gov/cuu/Units/binary.html>

Microsoft Learn: C# egész típusok és tartományok

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types>

Microsoft Learn: aritmetikai operátorok

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/arithmetic-operators>

Microsoft Learn: checked és unchecked

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked>

Források: technológia és párhuzamosság

Intel: Moore megfigyelésének története

<https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html>

V. Vinge (1993): The Coming Technological Singularity — NASA NTRS

<https://ntrs.nasa.gov/citations/19940022856>

G. M. Amdahl (1967): Validity of the Single Processor Approach...

<https://doi.org/10.1145/1465482.1465560>

J. L. Gustafson (1988): Reevaluating Amdahl's Law

<https://doi.org/10.1145/42411.42415>

M. J. Flynn (1972): Some Computer Organizations and Their Effectiveness

<https://doi.org/10.1109/TC.1972.5009071>

Képek és további szemléltető háttér

Áramköri fotó: Alexandre Debiève / Unsplash

<https://unsplash.com/photos/FO7JllwjOtU>

Szuperszámítógép-fotó: Texas Advanced Computing Center / Frontera

<https://tacc.utexas.edu/systems/all/>

LLNL: Introduction to Parallel Computing Tutorial

<https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>

A számítási példák és az ábrák az oktatási magyarázathoz készültek.